

Sprawozdanie z pracowni specjalistycznej

Systemy wbudowane

Tytuł: *Wyświetlanie liczb na wyświetlaczach 7-segmentowych. Kod BCD*

Wykonujący: *Monika Słowikowska, Filip Rutkowski*

Numer grupy PS: 9

Numer zespołu: 6

Numer pracy laboratoryjnej: 4

Studia dzienne

Kierunek: Informatyka

Semestr: VI

Prowadzący ćwiczenie: prof. dr hab. inż. Salauyou Valery

1. Zadanie 1

Wyświetl dwucyfrową liczbę $A = a_1a_0$ w kodzie BCD na ekranach HEX1 i HEX0, informuj w przypadku nieprawidłowych kodów numerycznych: 1010 - 1111, z następującym przyporządkowaniem pinów:

Wyprowadzeni a	Piny
a_1	SW[7:4]
a_0	SW[3:0]
error a_1	LEDR[9]
error a_0	LEDR[8]
[a_1]	HEX1
[a_0]	HEX0

Wartości przełączników SW [7: 0] mają być również wyświetlane na LEDR [7: 0].

Wskazówka: opracuj projekt decoder_hex_10 (input[3:0]x, output[0:6]h).

Kod w języku Verilog:

```

module showDecDigit (
    input [7:0] SW,
    output [9:0] LEDR,
    output [0:6] HEX0, HEX1);

    assign LEDR[7:0] = SW[7:0];

    wire [0:3] sw1 = SW[3:0];
    wire [0:3] sw2 = SW[7:4];
    reg led8 = 0;
    reg led9 = 0;

    decoder_hex_10 ex1(SW[3:0], HEX0);
    decoder_hex_10 ex2(SW[7:4], HEX1);

    assign LEDR[8] = led8;
    assign LEDR[9] = led9;

    always @(*)
    begin
        if(sw1 > 4'b1001)
            led8 = 1;
        else
            led8 = 0;

        if(sw2 > 4'b1001)
            led9 = 1;
        else
            led9 = 0;
    end
endmodule

```

Główny moduł

```

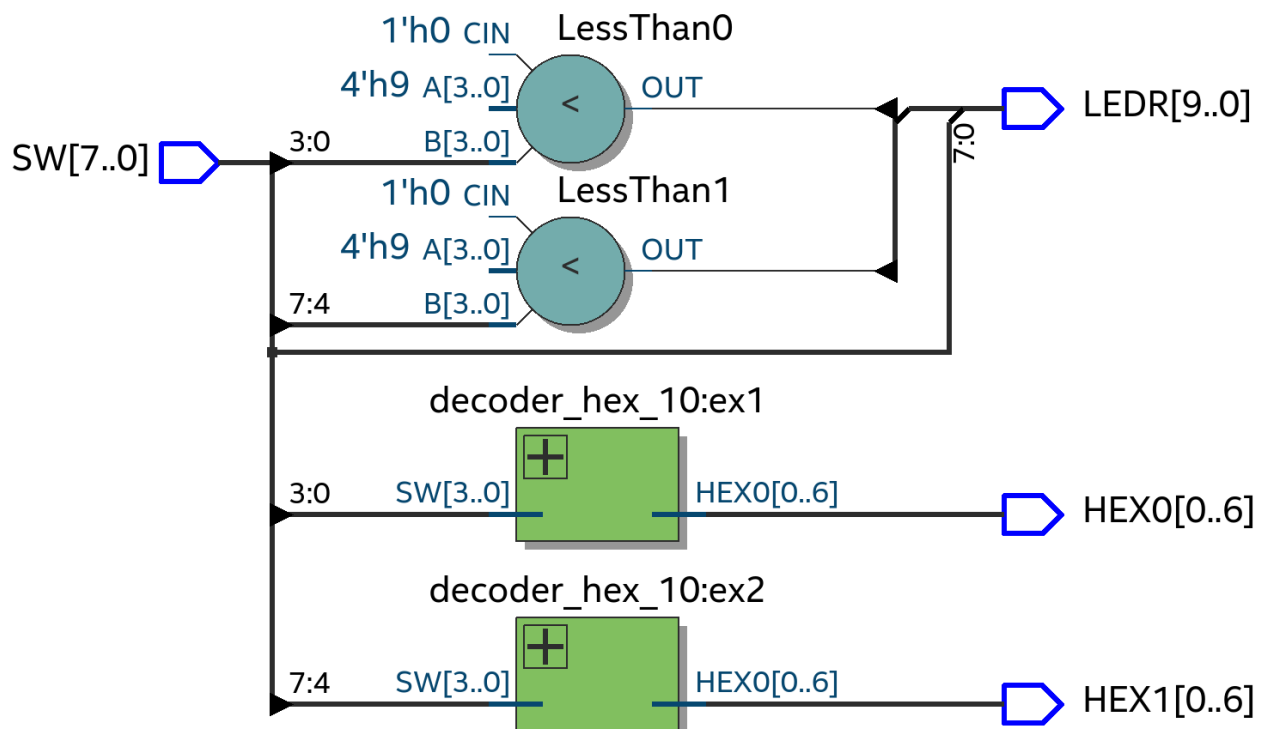
module decoder_hex_10 (
    input [3:0] SW,
    output reg [0:6] HEX0);

    always @(*)
        case(SW)
            4'b0000: HEX0 = ~7'b11111110;
            4'b0001: HEX0 = ~7'b01100000;
            4'b0010: HEX0 = ~7'b1101101;
            4'b0011: HEX0 = ~7'b1111001;
            4'b0100: HEX0 = ~7'b0110011;
            4'b0101: HEX0 = ~7'b1011011;
            4'b0110: HEX0 = ~7'b1011111;
            4'b0111: HEX0 = ~7'b1110000;
            4'b1000: HEX0 = ~7'b1111111;
            4'b1001: HEX0 = ~7'b1111011;
            4'b1010: HEX0 = ~7'b0000000;
            4'b1011: HEX0 = ~7'b0000000;
            4'b1100: HEX0 = ~7'b0000000;
            4'b1101: HEX0 = ~7'b0000000;
            4'b1110: HEX0 = ~7'b0000000;
            4'b1111: HEX0 = ~7'b0000000;
        endcase
endmodule

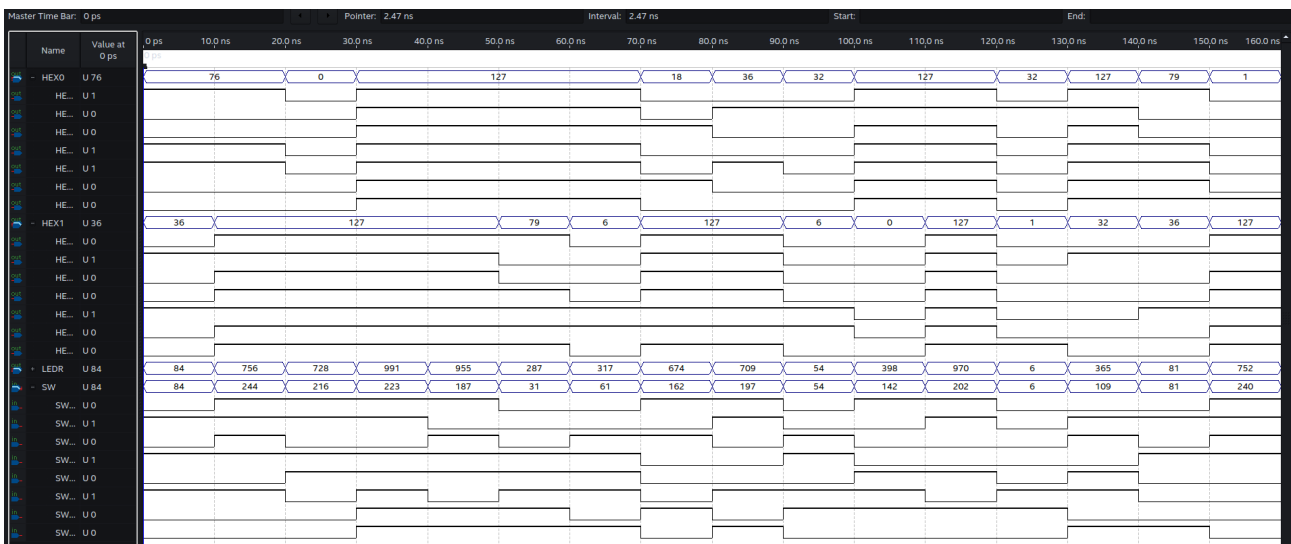
```

Moduł pomocniczy

Wyniki syntezy:



Wyniki symulacji:



Wnioski:

Za pomocą 8 przełączników (SW[0 - 7]) możemy wyświetlić dowolną liczbę dwucyfrową na wyświetlaczach HEX0 i HEX1. Przełączniki SW[0 - 3] służą do obsługi wyświetlacza HEX0, podczas gdy SW[4 - 7] odpowiadają za wyświetlacz HEX1. Jeżeli któryś z dekodów otrzyma sygnał przekraczający cyfrę 9 (1001) diody LED[8] lub LED[9] zasygnalizują, który z wyświetlaczy otrzymał nieprawidłową wartość, wyświetlacze natomiast nie będą pokazywać żadnej wartości.

2. Zadanie 2

Wyświetl dwucyfrową liczbę szesnastkową $B = b_1b_0$ na wyświetlaczach HEX1, HEX0.

Wskazówka: opracuj projekt `decoder_hex_16` (`input[3:0]x`, `output[0:6]h`).

Kod w języku Verilog:

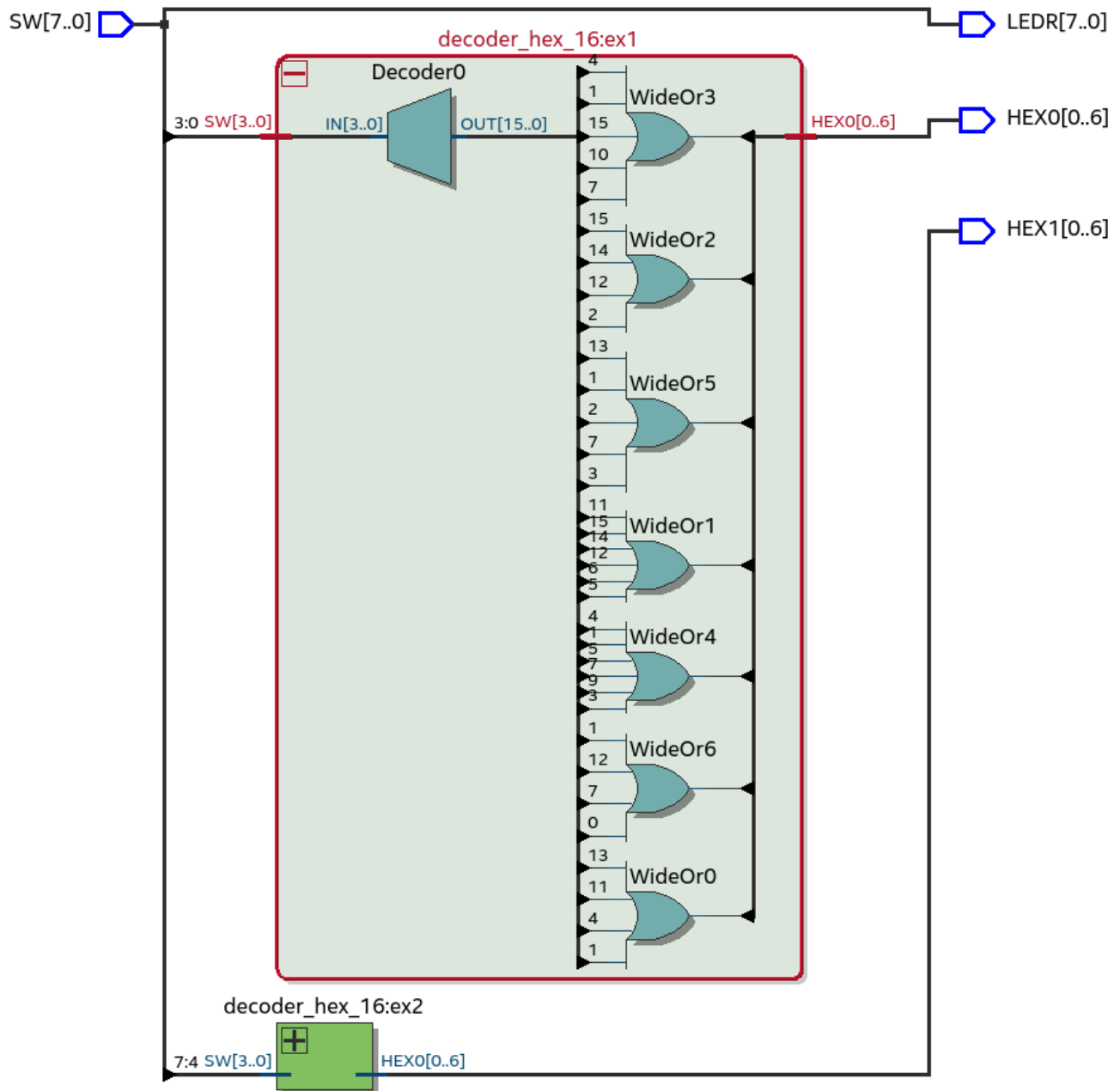
```
module showHexDigit(  
    input [7:0] SW,  
    output [7:0] LEDR,  
    output [0:6] HEX0, HEX1);  
  
    assign LEDR[7:0] = SW[7:0];  
  
    wire [0:3] sw1 = SW[3:0];  
    wire [0:3] sw2 = SW[7:4];  
  
    decoder_hex_16 ex1(SW[3:0], HEX0);  
    decoder_hex_16 ex2(SW[7:4], HEX1);  
  
endmodule
```

Moduł główny

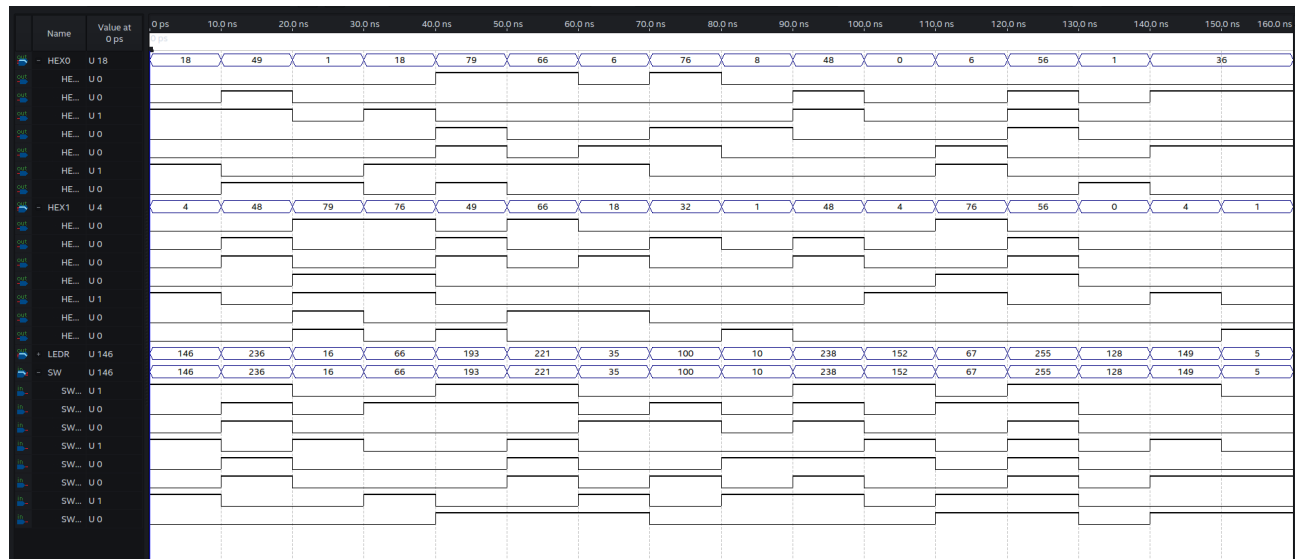
```
module decoder_hex_16 (  
    input [3:0] SW,  
    output reg [0:6] HEX0);  
  
    always @(*)  
        case(SW)  
            4'b0000: HEX0 = ~7'b1111110;  
            4'b0001: HEX0 = ~7'b0110000;  
            4'b0010: HEX0 = ~7'b1101101;  
            4'b0011: HEX0 = ~7'b1111001;  
            4'b0100: HEX0 = ~7'b0110011;  
            4'b0101: HEX0 = ~7'b1011011;  
            4'b0110: HEX0 = ~7'b1011111;  
            4'b0111: HEX0 = ~7'b1110000;  
            4'b1000: HEX0 = ~7'b1111111;  
            4'b1001: HEX0 = ~7'b1111011;  
            4'b1010: HEX0 = ~7'b1110111;  
            4'b1011: HEX0 = ~7'b0011111;  
            4'b1100: HEX0 = ~7'b1001110;  
            4'b1101: HEX0 = ~7'b0111101;  
            4'b1110: HEX0 = ~7'b1001111;  
            4'b1111: HEX0 = ~7'b1000111;  
        endcase  
  
endmodule
```

Moduł pomocniczy

Wyniki syntezy:



Wyniki symulacji:



Wnioski:

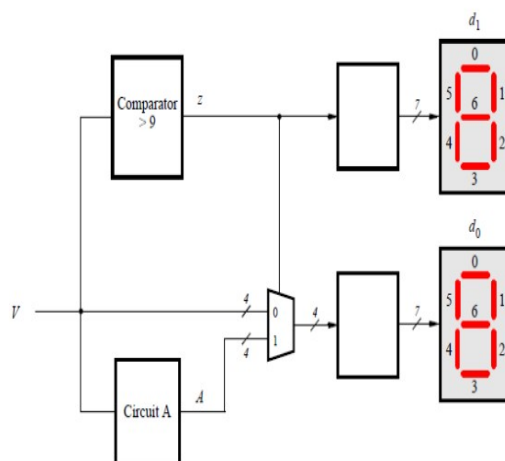
W tym zadaniu użyliśmy dwóch dekodерów szesnastkowych. Każdy z nich jest połączony do 4 przełączników dzięki, którym możemy zakodować 4 bitowe słowo, które zostanie przetłumaczone przez dekodер i wyświetlone na wyświetlaczu.

3. Zadanie 3

Opracuj projekt `binary_BCD_4_bits`, który konwertuje 4-bitową liczbę binarną $V = v_3v_2v_1v_0$ do postaci dwucyfrowej $D = d_1d_0$ w kodzie BCD (tab. 1, rys. 1).

Tabela 1. Wartości konwersji z kodu binarnego na dziesiętny

$v_3v_2v_1v_0$	d_1	d_0
0000	0	0
0001	0	1
0010	0	2
...	...	...
1001	0	9
1010	1	0
1011	1	1
1100	1	2
1101	1	3
1110	1	4
1111	1	5



Rys. 1. Struktura projektu układu konwersji z systemu binarnego na dziesiętny

Kod w języku Verilog:

```

module binary_BCD_4_bits (
    input [3:0] SW,
    output [0:6] HEX0, HEX1);

    wire [0:3] sw1 = SW[3:0];
    reg tens = 4'b0000;
    reg [0:3] unities;

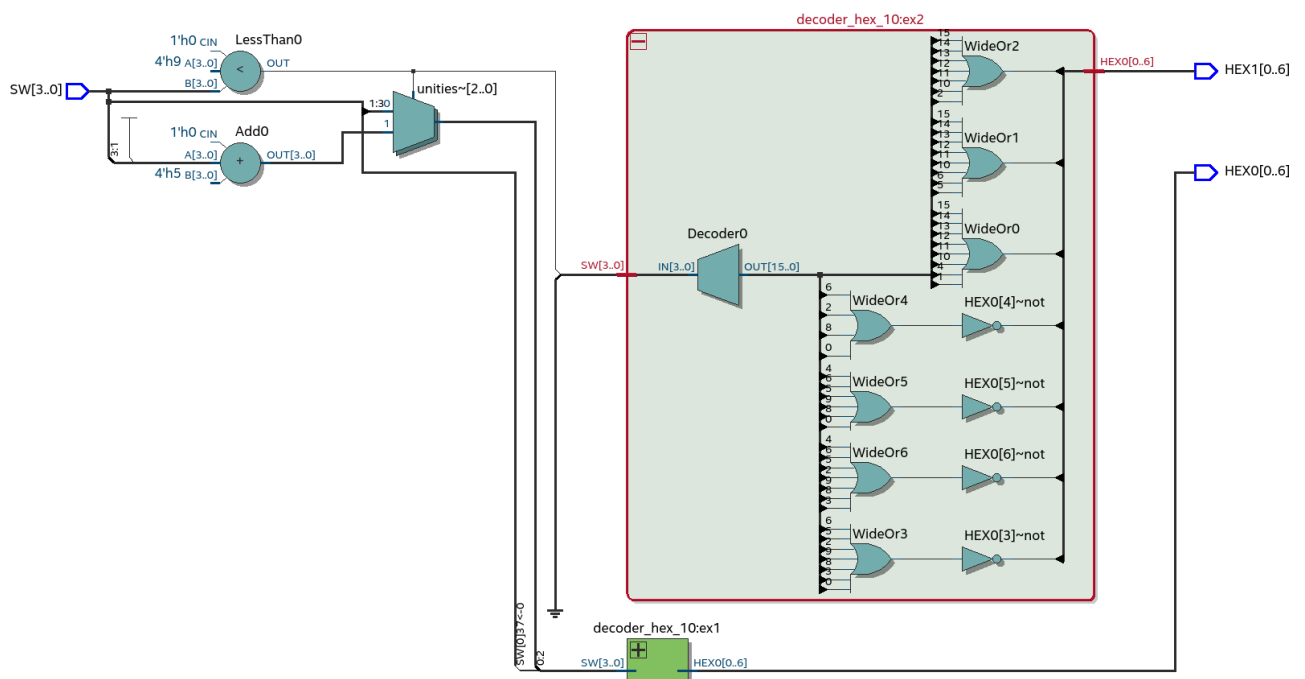
    decoder_hex_10 ex1(unities, HEX0);
    decoder_hex_10 ex2(tens, HEX1);

    always @(*)
    begin
        if(sw1 > 4'b1001)
            tens = 4'b0001;
        else
            tens = 4'b0000;

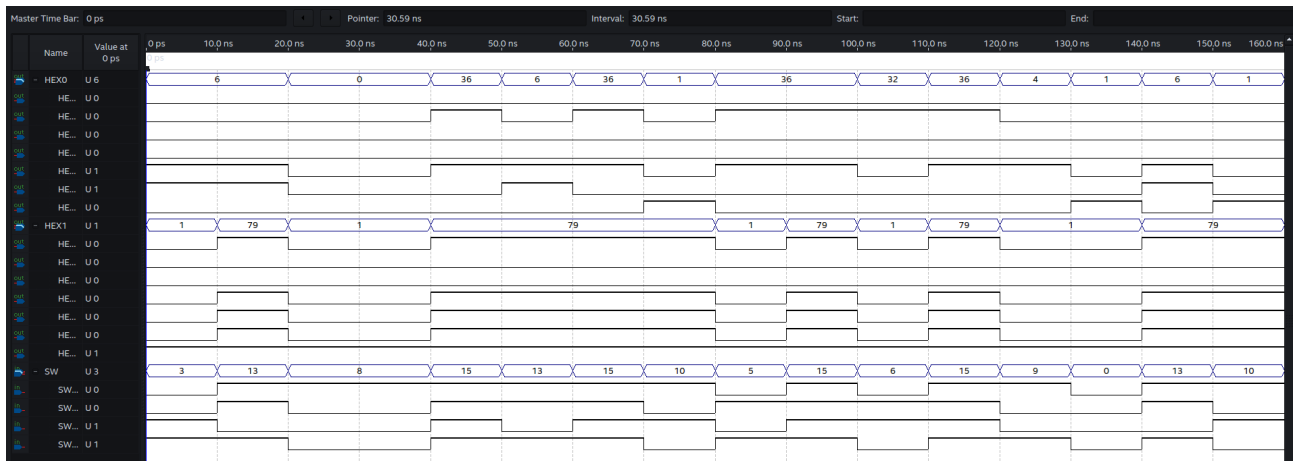
        if(tens == 4'b0001)
            unities = sw1 - 4'b1010;
        else
            unities = sw1;
    end
endmodule

```

Wyniki syntezy:



Wyniki symulacji:



Wnioski:

Korzystając z dekodera z zadania pierwszego, instrukcji warunkowych, instrukcji @always oraz dwóch zmiennych pomocniczych udało nam się stworzyć konwerter zamieniający czterobitowy kod BCD podany przez przełączniki na liczbę dziesiętną.

4. Zadanie 4

Opracuj moduł adder_BCD_2_digits, który dodaje dwie cyfry X i Y w kodzie BCD, i przeniesienie wejściowe cin, wynik jest reprezentowany jako dwucyfrowa suma $S = s_1s_0$ także w kodzie BCD.

Wskazówka: użyj układu (zamiast Circuit A), który generuje wartość najniższej cyfry kodu BCD dla sumy wartości $S > 10$.

Przypisanie pinów:

Wyprowadzeni a	Piny
X	SW[7:4], HEX5
Y	SW[3:0], HEX3
cin	SW[8]
S0	HEX0
S1	HEX1
error	LEDR[9]
SW[7:0]	LEDR[7:0]

Kod w języku Verilog:

```
module adder_BCD_2_digits (
    input [8:0] SW,
    output [9:0] LEDR,
    output [0:6] HEX0, HEX1, HEX3, HEX5);

    reg error = 0;
    assign LEDR[9] = error;
    assign LEDR[8] = 0;
    assign LEDR[7:0] = SW[7:0];

    decoder_hex_10 ex1(SW[7:4], HEX5);
    decoder_hex_10 ex2(SW[3:0], HEX3);

    wire [0:3] x = SW[7:4];
    wire [0:3] y = SW[3:0];
    wire cin = SW[8];
    wire [0:4] s = x + y + cin;

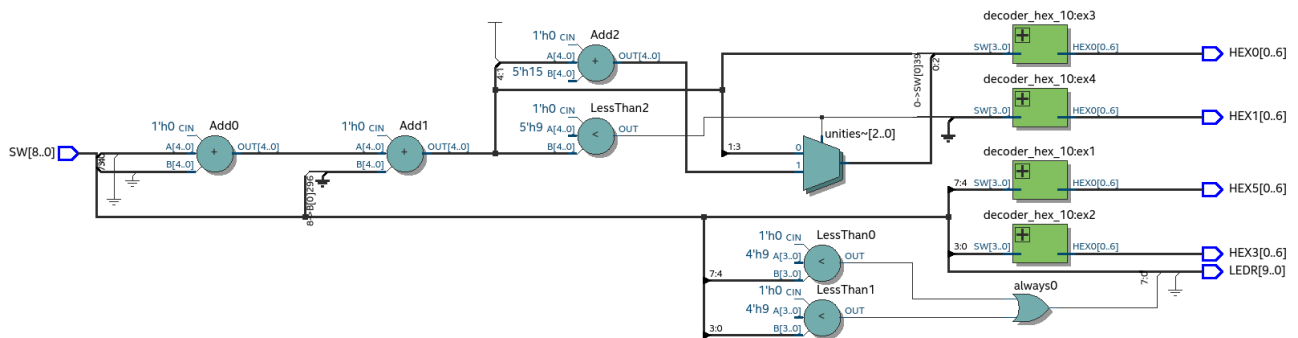
    reg tens = 4'b0000;
    reg [0:3] unities;

    decoder_hex_10 ex3(unities, HEX0);
    decoder_hex_10 ex4(tens, HEX1);

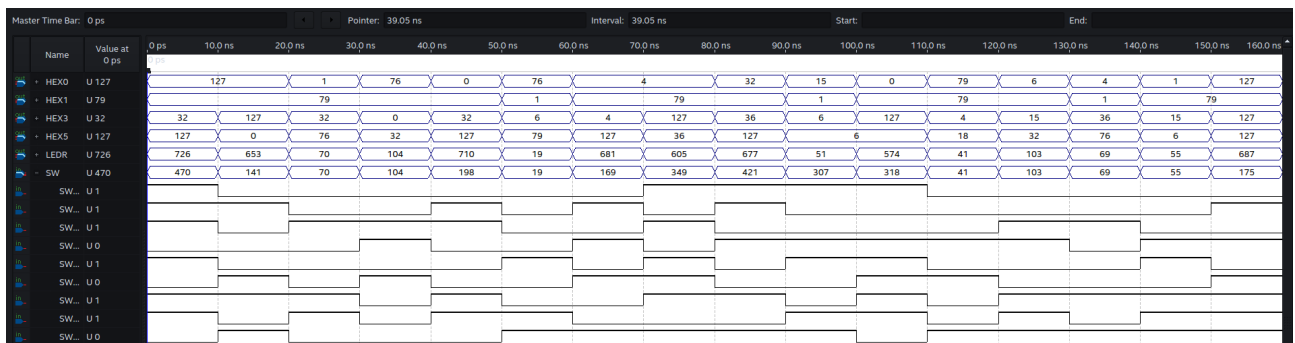
    always @(*)
        begin
            if(x > 4'b1001 || y > 4'b1001)
                error = 1;
            else
                error = 0;

            if(s > 4'b1001)
                begin
                    tens = 4'b0001;
                    unities = s - 4'b1010;
                end
            else
                begin
                    tens = 4'b0000;
                    unities = s;
                end
            end
        end
endmodule
```

Wyniki syntezy:



Wyniki symulacji:



Wnioski:

Wykorzystując wiedzę i moduły z poprzednich zadań utworzyliśmy projekt dodający dwie liczby podawane przez przełączniki i wyświetlane na wyświetlaczach HEX3 i HEX5. Suma tych liczb oraz przeniesienia podawanego przez przełącznik SW[8] zostaje wyświetlona na HEX0 i HEX1 w postaci dziesiętnej.

5. Opracuj projekt adder_BCD_2_digits_b, który rozwiązuje poprzedni problem algorytmicznie zgodnie z następującym pseudokodem:

```
1   $T_0 = A + B + c_0$ 
2  if ( $T_0 > 9$ ) then
3       $Z_0 = 10$ ;
4       $c_1 = 1$ ;
5  else
6       $Z_0 = 0$ ;
7       $c_1 = 0$ ;
8  end if
9   $S_0 = T_0 - Z_0$ 
10  $S_1 = c_1$ 
```

Kod w języku Verilog:

```
module adder_BCD_2_digits_b (
    input [8:0] SW,
    output [9:0] LEDR,
    output [0:6] HEX0, HEX1, HEX3, HEX5);

    reg error = 0;
    assign LEDR[9] = error;
    assign LEDR[8] = 0;
    assign LEDR[7:0] = SW[7:0];

    decoder_hex_10 ex1(SW[7:4], HEX5);
    decoder_hex_10 ex2(SW[3:0], HEX3);

    wire [0:3] A = SW[7:4];
    wire [0:3] B = SW[3:0];
    wire c0 = SW[8];
    reg [0:3] c1;
    reg [0:3] z0;
    wire [0:4] T0 = A + B + c0;

    reg[0:3] S0, S1;

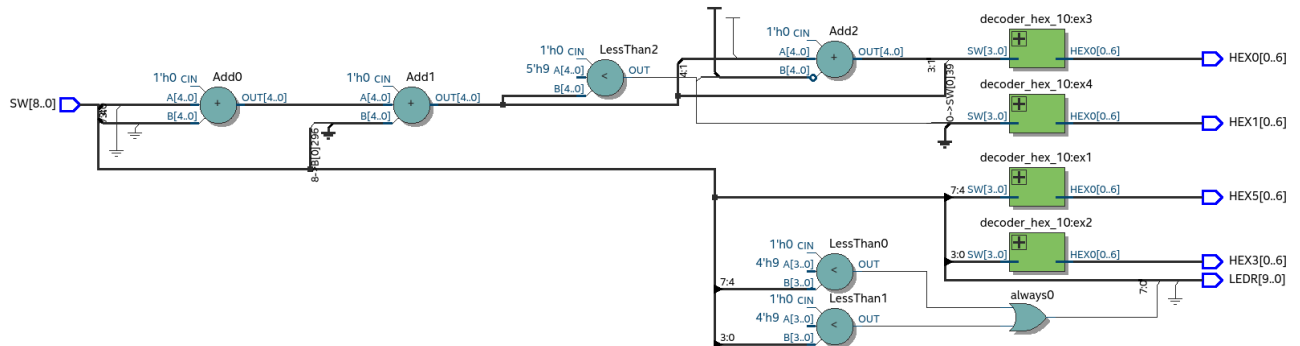
    decoder_hex_10 ex3(S0, HEX0);
    decoder_hex_10 ex4(S1, HEX1);

    always @(*)
        begin
            if(A > 9 || B > 9)
                error = 1;
            else
                error = 0;

            if(T0 > 9)
                begin
                    z0 = 10;
                    c1 = 1;
                end
            else
                begin
                    z0 = 0;
                    c1 = 0;
                end

            S0 = T0 - z0;
            S1 = c1;
        end
endmodule
```

Wyniki syntezy:



Wyniki symulacji:

Master Time Bar: 0 ps			Pointer: 15.77 ns			Interval: 15.77 ns			Start:			End:							
			0 ps	10.0 ns	20.0 ns	30.0 ns	40.0 ns	50.0 ns	60.0 ns	70.0 ns	80.0 ns	90.0 ns	100.0 ns	110.0 ns	120.0 ns	130.0 ns	140.0 ns	150.0 ns	160.0 ns
	Name	Value at 0 ps																	
	HEX0 U 76	76	127	79	0	76	36	15	36	1	79	4	6	79	1	0			
	HEX1 U 79	79	1	79	1	79	1	79	1	1	1	1	1	1	1	1			
	HEX3 U 36	36	4	1	32	18	127	36	4	15	127	1	15	127	6	4	0		
	HEX5 U 4	4	127	79	127	36	18	32	0	127	79	127	127	0	79	1			
	SW U 149	149	233	16	198	194	90	37	105	135	236	16	199	254	131	25	8		

Wnioski:

W powyższym zadaniu opracowaliśmy moduł, który, analogicznie do zadania 4, dodaje dwie cyfry w kodzie BCD oraz przeniesienie wejściowe cin, a następnie wyświetla wynik również w kodzie BCD.

Tym co różni to rozwiązanie od zastosowanego w poprzednim zadaniu jest użyty algorytm. Po dodaniu do siebie obu liczb, porównujemy ich sumę (zmienna T0) do liczby 9 w celu określenia, czy rzeczona suma jest w zapisie dziesiętnym jedno- czy dwucyfrowa. W przypadku dwucyfrowego wyniku ustawiamy wartość Z0 na 10, a c1 na 1. Po wyjściu z instrukcji warunkowej obliczamy cyfrę jedności (S0) i dziesiątek (S1), które zostaną następnie podane do dekodery i ostatecznie wyświetlone na wyświetlaczach HEX0 i HEX1.